Kế Hoạch Triển Khai Chi Tiết Lab 3: Computer Store Management System

(Dựa trên cấu trúc và tư duy kiến trúc của project `sqlite_intro`)

1. Phân tích Tư duy & Cấu trúc từ `sqlite_intro`

Trong project mẫu , ứng dụng được tổ chức theo kiến trúc phân tầng rõ ràng kết hợp **SQLite (`sqflite`)** và **Riverpod (`flutter_riverpod` + `riverpod_annotation`)**:

1. **`lib/model/`** :
   - Chứa các lớp dữ liệu (Data Model) với các thuộc tính `final`.
   - Sử dụng phương thức `toJson()` để chuyển Object thành `Map<String, dynamic>` khi lưu xuống SQLite (`db.insert` , `db.update`).
   - Sử dụng `factory Model.fromJson(Map<String, dynamic> json)` để parse dữ liệu từ kết quả truy vấn SQLite (`List<Map<String, Object?>>`) sang Object Dart.
2. **`lib/db/database_helper.dart`** :
   - Áp dụng **Singleton Pattern** (`DatabaseHelper._privateConstructor()` và `DatabaseHelper.instance`).
   - Quản lý kết nối qua getter `Future<Database> get database async` dùng toán tử `??=`.
   - Hỗ trợ đa nền tảng: kiểm tra `if (kIsWeb)` để chuyển `databaseFactory = databaseFactoryFfiWeb` , hoặc lấy đường dẫn thực tế bằng `getDatabasesPath()` + `join()` trên Mobile.
   - Đóng gói toàn bộ câu lệnh SQL (`CREATE TABLE` , `INSERT` , `UPDATE` , `DELETE` , `SELECT` , `JOIN` , `GROUP BY` , `Aggregate`) tại đây.
3. **`lib/providers/`** :
   - Sử dụng `@riverpod` class kế thừa `_$...Notifier` (`AsyncNotifier`).
   - Hàm `build()` trả về `Future<T>` gọi trực tiếp từ `DatabaseHelper.instance`.
   - Các hàm thay đổi dữ liệu (thêm, sửa, xóa, lọc, tìm kiếm) sau khi gọi `DatabaseHelper.instance` sẽ gọi `ref.invalidateSelf(); await future;` (hoặc cập nhật `state = AsyncData(...)`) để UI tự động đồng bộ mà không cần quản lý `setState` thủ công rời rạc.

4. **`lib/ui/`**:
    - Các màn hình kế thừa `ConsumerStatefulWidget` & `ConsumerState`.
    - Lắng nghe dữ liệu bằng `final asyncVal = ref.watch(...Provider);` và render 3 trạng thái thông qua `.when(data: ..., error: ..., loading: ...)`.
    - Gọi các hành động thông qua `ref.read(...Provider.notifier)`.

Cấu trúc thư mục mục tiêu cho Lab 3

```
text
lib/
├── main.dart
├── db/
│   └── database_helper.dart         # Singleton quản lý ComputerStoreDB.db & toàn
bộ SQL
├── model/
│   ├── brand.dart                   # Model Brand (kèm totalProducts từ LEFT JOIN)
│   ├── category.dart                # Model Category (kèm totalProducts từ LEFT
JOIN)
│   ├── computer.dart                # Model Computer (kèm brandName, categoryName
& các getter tính toán)
│   └── store_statistics.dart        # Các Model chứa kết quả thống kê (Task 6)
├── providers/
│   ├── brand_notifier.dart          # Riverpod AsyncNotifier quản lý Brand +
Search Brand
│   ├── brand_notifier.g.dart
│   ├── category_notifier.dart       # Riverpod AsyncNotifier quản lý Category +
Search Category
│   ├── category_notifier.g.dart
│   ├── computer_notifier.dart       # Riverpod AsyncNotifier quản lý Computer +
Search/Filter/Sort
│   ├── computer_notifier.g.dart
│   ├── statistics_notifier.dart     # Riverpod AsyncNotifier quản lý dữ liệu Thống
kê
│   └── statistics_notifier.g.dart
└── ui/
    ├── main_navigation_screen.dart  # Màn hình chính chứa BottomNavigationBar 6
tabs
    ├── store_screen.dart            # Tab 1: Home (Banner, Search, Filter, Sort,
Product List)
    ├── computer_form_screen.dart    # Tab 2 & Màn hình Edit: Thêm / Sửa Computer
    ├── computer_detail_screen.dart  # Màn hình Chi tiết Computer + Business
Calculations
    ├── brand_list_screen.dart       # Tab 3: Quản lý Brand (List, Search,
Add/Edit/Delete Dialog)
    ├── category_list_screen.dart    # Tab 4: Quản lý Category (List, Search,
Add/Edit/Delete Dialog)
    ├── statistics_screen.dart       # Tab 5: Thống kê kinh doanh từ SQLite
Aggregate
    └── featured_screen.dart         # Tab 6: Danh sách sản phẩm Featured
```

Bước 1: Cấu hình `pubspec.yaml` và Assets (Task 1 & Task 2.6)

Giải thích

- Giữ nguyên bộ thư viện chuẩn của gồm `sqflite`, `path`, `sqflite_common_ffi_web`, `flutter_riverpod`, `riverpod_annotation`.
- Bổ sung khai báo thư mục `assets/images/` để hiển thị **Promotional Banner** ở đầu màn hình Store (Task 2.6).

Thao tác

1. Tạo thư mục `assets/images/` ở thư mục gốc của project và đặt 1 ảnh banner (ví dụ `assets/images/promo_banner.png`).
2. Cập nhật phần `flutter:` trong :

```
yaml
flutter:
  uses-material-design: true
  assets:
    - assets/images/
```

Bước 2: Xây dựng tầng `lib/model/` (Task 2, 3, 4, 6)

Giải thích

Theo đúng chuẩn của , mỗi model đều có `toJson()` (chỉ chứa các cột thực sự nằm trong bảng SQLite để `insert`/`update`) và `fromJson()` (đọc cả các cột của bảng lẫn các cột ảo sinh ra từ câu lệnh `JOIN` như `totalProducts`, `brand_name`, `category_name`).

2.1. File `lib/model/brand.dart`

```
dart
class Brand {
  final int? id;
  final String brandName;
  final String country;
  final int foundedYear;
  final int totalProducts; // Được tính từ SQL LEFT JOIN + COUNT(c.id)

  Brand({
    this.id,
    required this.brandName,
    required this.country,
    required this.foundedYear,
    this.totalProducts = 0,
  });

  Map<String, dynamic> toJson() {
    return {
      if (id != null) 'id': id,
      'brand_name': brandName,
      'country': country,
      'founded_year': foundedYear,
    };
  }

  factory Brand.fromJson(Map<String, dynamic> json) {
    return Brand(
      id: json['id'] as int?,
      brandName: json['brand_name'] as String,
      country: json['country'] as String,
      foundedYear: json['founded_year'] as int,
      totalProducts: (json['totalProducts'] as num?)?.toInt() ?? 0,
    );
  }
}
```

2.2. File `lib/model/category.dart`

```
dart
class CategoryModel {
  final int? id;
  final String categoryName;
  final int totalProducts; // Được tính từ SQL LEFT JOIN + COUNT(c.id)

  CategoryModel({
    this.id,
    required this.categoryName,
    this.totalProducts = 0,
  });

  Map<String, dynamic> toJson() {
    return {
      if (id != null) 'id': id,
      'category_name': categoryName,
    };
  }

  factory CategoryModel.fromJson(Map<String, dynamic> json) {
    return CategoryModel(
      id: json['id'] as int?,
      categoryName: json['category_name'] as String,
      totalProducts: (json['totalProducts'] as num?)?.toInt() ?? 0,
    );
  }
}
```

2.3. File `lib/model/computer.dart`

> IMPORTANT
>
> Ở Task 4.5, đề bài yêu cầu 3 giá trị **Revenue**, **Inventory Value**, và **Total Units** không được lưu trong database mà phải tính toán trực tiếp trong Flutter. Ta định nghĩa chúng dưới dạng **computed getters** ngay trong class `Computer`.

```dart
class Computer {
  final int? id;
  final String productName;
  final int brandId;
  final int categoryId;
  final String cpu;
  final int ram;
  final int storage;
  final double price;
  final int stock;
  final int soldQuantity;
  final bool isFeatured;

  // Các trường lấy từ INNER JOIN với bảng brands và categories
  final String? brandName;
  final String? categoryName;

  Computer({
    this.id,
    required this.productName,
    required this.brandId,
    required this.categoryId,
    required this.cpu,
    required this.ram,
    required this.storage,
    required this.price,
    required this.stock,
    required this.soldQuantity,
    required this.isFeatured,
    this.brandName,
    this.categoryName,
  });

  // Business Calculations (Task 4.5 - tính trực tiếp trong Flutter)
  double get revenue => price * soldQuantity;
  double get inventoryValue => price * stock;
  int get totalUnits => stock + soldQuantity;

  Map<String, dynamic> toJson() {
    return {
      if (id != null) 'id': id,
      'product_name': productName,
      'brand_id': brandId,
      'category_id': categoryId,
      'cpu': cpu,
```

```
      'ram': ram,
      'storage': storage,
      'price': price,
      'stock': stock,
      'sold_quantity': soldQuantity,
      'is_featured': isFeatured ? 1 : 0, // SQLite lưu bool dưới dạng INTEGER 0/1
    };
  }

  factory Computer.fromJson(Map<String, dynamic> json) {
    return Computer(
      id: json['id'] as int?,
      productName: json['product_name'] as String,
      brandId: (json['brand_id'] as num?)?.toInt() ?? 0,
      categoryId: (json['category_id'] as num?)?.toInt() ?? 0,
      cpu: json['cpu'] as String,
      ram: (json['ram'] as num).toInt(),
      storage: (json['storage'] as num).toInt(),
      price: (json['price'] as num).toDouble(),
      stock: (json['stock'] as num).toInt(),
      soldQuantity: (json['sold_quantity'] as num).toInt(),
      isFeatured: (json['is_featured'] as num).toInt() == 1,
      brandName: json['brand_name'] as String?,
      categoryName: json['category_name'] as String?,
    );
  }
}
```

2.4. File `lib/model/store_statistics.dart`

Chứa cấu trúc dữ liệu cho toàn bộ màn hình **Statistics** (Task 6) và bộ lọc nâng cao (Task 5):

```dart
class GeneralStats {
  final int totalProducts;
  final double totalRevenue;
  final int totalSoldQuantity;
  final double averagePrice;
  final double totalInventoryValue;

  GeneralStats({
    required this.totalProducts,
    required this.totalRevenue,
    required this.totalSoldQuantity,
    required this.averagePrice,
    required this.totalInventoryValue,
  });
}

class BrandStatItem {
  final String brandName;
  final int totalProducts;
  final int totalSold;
  final double revenue;
  final double inventoryValue;

  BrandStatItem({
    required this.brandName,
    required this.totalProducts,
    required this.totalSold,
    required this.revenue,
    required this.inventoryValue,
  });

  factory BrandStatItem.fromJson(Map<String, dynamic> json) {
    return BrandStatItem(
      brandName: json['brand_name'] as String,
      totalProducts: (json['TotalProducts'] as num?)?.toInt() ?? 0,
      totalSold: (json['TotalSold'] as num?)?.toInt() ?? 0,
      revenue: (json['Revenue'] as num?)?.toDouble() ?? 0.0,
      inventoryValue: (json['InventoryValue'] as num?)?.toDouble() ?? 0.0,
    );
  }
}

class CategoryStatItem {
  final String categoryName;
  final int totalProducts;
```

```
  final double revenue;
  final int totalSold;

  CategoryStatItem({
    required this.categoryName,
    required this.totalProducts,
    required this.revenue,
    required this.totalSold,
  });

  factory CategoryStatItem.fromJson(Map<String, dynamic> json) {
    return CategoryStatItem(
      categoryName: json['category_name'] as String,
      totalProducts: (json['totalProducts'] as num?)?.toInt() ?? 0,
      revenue: (json['totalRevenue'] as num?)?.toDouble() ?? 0.0,
      totalSold: (json['totalSold'] as num?)?.toInt() ?? 0,
    );
  }
}

class FeaturedStats {
  final int totalFeaturedProducts;
  final double totalFeaturedRevenue;
  final int totalFeaturedSoldQuantity;

  FeaturedStats({
    required this.totalFeaturedProducts,
    required this.totalFeaturedRevenue,
    required this.totalFeaturedSoldQuantity,
  });
}

class DashboardStatistics {
  final GeneralStats general;
  final List<BrandStatItem> byBrand;
  final List<CategoryStatItem> byCategory;
  final Map<String, dynamic>? bestSellingProduct;
  final Map<String, dynamic>? mostExpensiveProduct;
  final List<Map<String, dynamic>> lowStockProducts;
  final Map<String, dynamic>? topBrand;
  final FeaturedStats featuredStats;

  DashboardStatistics({
    required this.general,
    required this.byBrand,
    required this.byCategory,
    this.bestSellingProduct,
```

```
    this.mostExpensiveProduct,
    required this.lowStockProducts,
    this.topBrand,
    required this.featuredStats,
  });
}
```

Bước 3: Nâng cấp `lib/db/database_helper.dart` (Task 1 → Task 6)

Giải thích

Giữ nguyên cấu trúc Singleton và cơ chế khởi tạo Web/Mobile của , ta mở rộng:

1. Đổi `_databaseName = 'ComputerStoreDB.db'`.
2. Bật `PRAGMA foreign_keys = ON` trong `onConfigure`.
3. Tạo 3 bảng `brands`, `categories`, `computers` trong `_onCreate` và gọi `_seedSampleData(db)` để chèn sẵn **5 Brands**, **3 Categories**, và **15 Computers** (Task 1).
4. Cung cấp đầy đủ các phương thức CRUD, kiểm tra trùng tên (Unique validation), kiểm tra khóa ngoại trước khi xóa (`COUNT(*)`), truy vấn `INNER JOIN` / `LEFT JOIN`, lọc động nhiều điều kiện (`WHERE ... AND ...`) và thống kê `SUM`, `COUNT`, `AVG`, `GROUP BY`.

Code chi tiết `lib/db/database_helper.dart`

```
dart
import 'package:flutter/foundation.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

import '../model/brand.dart';
import '../model/category.dart';
import '../model/computer.dart';
import '../model/store_statistics.dart';

class DatabaseHelper {
  static const _databaseName = 'ComputerStoreDB.db';
  static const _brandTable = 'brands';
  static const _categoryTable = 'categories';
  static const _computerTable = 'computers';

  // Singleton pattern (giữ đúng chuẩn sqlite_intro)
  DatabaseHelper._privateConstructor();
  static final DatabaseHelper instance = DatabaseHelper._privateConstructor();

  static Database? _database;
  Future<Database> get database async {
    _database ??= await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    if (kIsWeb) {
      databaseFactory = databaseFactoryFfiWeb;
      return openDatabase(
        _databaseName,
        version: 1,
        onConfigure: _onConfigure,
        onCreate: _onCreate,
      );
    }
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, _databaseName);
    return openDatabase(
      path,
      version: 1,
      onConfigure: _onConfigure,
      onCreate: _onCreate,
```

```
    );
  }

  Future<void> _onConfigure(Database db) async {
    await db.execute('PRAGMA foreign_keys = ON');
  }

  Future<void> _onCreate(Database db, int version) async {
    await db.execute('''
      CREATE TABLE $_brandTable (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        brand_name TEXT NOT NULL,
        country TEXT NOT NULL,
        founded_year INTEGER NOT NULL
      )
    ''');

    await db.execute('''
      CREATE TABLE $_categoryTable (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        category_name TEXT NOT NULL
      )
    ''');

    await db.execute('''
      CREATE TABLE $_computerTable (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        product_name TEXT NOT NULL,
        brand_id INTEGER NOT NULL,
        category_id INTEGER NOT NULL,
        cpu TEXT NOT NULL,
        ram INTEGER NOT NULL,
        storage INTEGER NOT NULL,
        price REAL NOT NULL,
        stock INTEGER NOT NULL,
        sold_quantity INTEGER NOT NULL,
        is_featured INTEGER NOT NULL DEFAULT 0,
        FOREIGN KEY (brand_id) REFERENCES $_brandTable (id),
        FOREIGN KEY (category_id) REFERENCES $_categoryTable (id)
      )
    ''');

    await _seedSampleData(db);
  }

  // Task 1: Insert sample data (Minimum: 5 Brands, 3 Categories, 15 Computers)
  Future<void> _seedSampleData(Database db) async {
```

```
    // 5 Brands
    final brands = [
      {'brand_name': 'ASUS', 'country': 'Taiwan', 'founded_year': 1989},
      {'brand_name': 'Dell', 'country': 'USA', 'founded_year': 1984},
      {'brand_name': 'HP', 'country': 'USA', 'founded_year': 1939},
      {'brand_name': 'Acer', 'country': 'Taiwan', 'founded_year': 1976},
      {'brand_name': 'Lenovo', 'country': 'China', 'founded_year': 1984},
    ];
    for (final b in brands) {
      await db.insert(_brandTable, b);
    }

    // 3 Categories
    final categories = [
      {'category_name': 'Laptop'},
      {'category_name': 'Desktop'},
      {'category_name': 'Component'},
    ];
    for (final c in categories) {
      await db.insert(_categoryTable, c);
    }

    // 15 Computers
    final computers = [
      {
        'product_name': 'ROG Strix G16',
        'brand_id': 1,
        'category_id': 1,
        'cpu': 'Intel Core i7',
        'ram': 16,
        'storage': 1024,
        'price': 1250.0,
        'stock': 15,
        'sold_quantity': 32,
        'is_featured': 1,
      },
      {
        'product_name': 'Vivobook S15',
        'brand_id': 1,
        'category_id': 1,
        'cpu': 'Intel Core i5',
        'ram': 16,
        'storage': 512,
        'price': 780.0,
        'stock': 20,
        'sold_quantity': 45,
```

```
    'is_featured': 0,
  },
  {
    'product_name': 'TUF Gaming A15',
    'brand_id': 1,
    'category_id': 1,
    'cpu': 'AMD Ryzen 7',
    'ram': 16,
    'storage': 512,
    'price': 950.0,
    'stock': 8,
    'sold_quantity': 28,
    'is_featured': 1,
  },
  {
    'product_name': 'ROG Maximus Z790 Board',
    'brand_id': 1,
    'category_id': 3,
    'cpu': 'Socket LGA1700',
    'ram': 64,
    'storage': 256,
    'price': 420.0,
    'stock': 4,
    'sold_quantity': 12,
    'is_featured': 0,
  },
  {
    'product_name': 'Dell XPS 15',
    'brand_id': 2,
    'category_id': 1,
    'cpu': 'Intel Core i9',
    'ram': 32,
    'storage': 1024,
    'price': 1850.0,
    'stock': 6,
    'sold_quantity': 19,
    'is_featured': 1,
  },
  {
    'product_name': 'Dell Inspiron 14',
    'brand_id': 2,
    'category_id': 1,
    'cpu': 'Intel Core i5',
    'ram': 8,
    'storage': 256,
    'price': 480.0,
```

```
    'stock': 25,
    'sold_quantity': 60,
    'is_featured': 0,
  },
  {
    'product_name': 'OptiPlex 7010 Tower',
    'brand_id': 2,
    'category_id': 2,
    'cpu': 'Intel Core i7',
    'ram': 16,
    'storage': 512,
    'price': 890.0,
    'stock': 12,
    'sold_quantity': 22,
    'is_featured': 0,
  },
  {
    'product_name': 'HP Spectre x360',
    'brand_id': 3,
    'category_id': 1,
    'cpu': 'Intel Core i7',
    'ram': 16,
    'storage': 1024,
    'price': 1400.0,
    'stock': 3,
    'sold_quantity': 18,
    'is_featured': 1,
  },
  {
    'product_name': 'HP Pavilion Gaming Desktop',
    'brand_id': 3,
    'category_id': 2,
    'cpu': 'AMD Ryzen 5',
    'ram': 16,
    'storage': 512,
    'price': 750.0,
    'stock': 10,
    'sold_quantity': 25,
    'is_featured': 0,
  },
  {
    'product_name': 'HP Omen 45L',
    'brand_id': 3,
    'category_id': 2,
    'cpu': 'Intel Core i9',
    'ram': 64,
```

```
    'storage': 2048,
    'price': 2400.0,
    'stock': 2,
    'sold_quantity': 9,
    'is_featured': 1,
  },
  {
    'product_name': 'Acer Nitro 5',
    'brand_id': 4,
    'category_id': 1,
    'cpu': 'Intel Core i5',
    'ram': 8,
    'storage': 512,
    'price': 690.0,
    'stock': 18,
    'sold_quantity': 50,
    'is_featured': 1,
  },
  {
    'product_name': 'Acer Predator Orion 3000',
    'brand_id': 4,
    'category_id': 2,
    'cpu': 'Intel Core i7',
    'ram': 32,
    'storage': 1024,
    'price': 1350.0,
    'stock': 0,
    'sold_quantity': 15,
    'is_featured': 0,
  },
  {
    'product_name': 'Acer Predator Bifrost GPU Kit',
    'brand_id': 4,
    'category_id': 3,
    'cpu': 'Intel Arc A770',
    'ram': 16,
    'storage': 256,
    'price': 390.0,
    'stock': 7,
    'sold_quantity': 14,
    'is_featured': 0,
  },
  {
    'product_name': 'Lenovo ThinkPad X1 Carbon',
    'brand_id': 5,
    'category_id': 1,
```

```
        'cpu': 'Intel Core i7',
        'ram': 32,
        'storage': 1024,
        'price': 1650.0,
        'stock': 9,
        'sold_quantity': 35,
        'is_featured': 1,
      },
      {
        'product_name': 'Lenovo Legion Tower 5i',
        'brand_id': 5,
        'category_id': 2,
        'cpu': 'Intel Core i7',
        'ram': 32,
        'storage': 2048,
        'price': 1550.0,
        'stock': 4,
        'sold_quantity': 21,
        'is_featured': 1,
      },
    ];
    for (final comp in computers) {
      await db.insert(_computerTable, comp);
    }
  }

  // ============================================================================
  // TASK 2: BRAND MANAGEMENT
  // ============================================================================

  // 2.1 & 2.5: Lấy danh sách Brand kèm số lượng sản phẩm (hỗ trợ tìm kiếm theo
tên/quốc gia)
  Future<List<Brand>> getBrands({String? keyword}) async {
    final db = await database;
    String whereClause = '';
    List<Object?> args = [];

    if (keyword != null && keyword.trim().isNotEmpty) {
      final kw = '%${keyword.trim()}%';
      whereClause = 'WHERE b.brand_name LIKE ? COLLATE NOCASE OR b.country LIKE ?
COLLATE NOCASE';
      args = [kw, kw];
    }

    final maps = await db.rawQuery('''
      SELECT
        b.id,
```

```
      b.brand_name,
      b.country,
      b.founded_year,
      COUNT(c.id) AS totalProducts
    FROM $_brandTable b
    LEFT JOIN $_computerTable c ON b.id = c.brand_id
    $whereClause
    GROUP BY b.id
    ORDER BY b.brand_name ASC
  ''', args);

  return [for (final map in maps) Brand.fromJson(map)];
}

Future<bool> isBrandNameDuplicate(String name, {int? excludeId}) async {
  final db = await database;
  final maps = await db.query(
    _brandTable,
    where: excludeId != null
        ? 'LOWER(brand_name) = LOWER(?) AND id != ?'
        : 'LOWER(brand_name) = LOWER(?)',
    whereArgs: excludeId != null ? [name.trim(), excludeId] : [name.trim()],
  );
  return maps.isNotEmpty;
}

Future<int> insertBrand(Brand brand) async {
  final db = await database;
  return db.insert(_brandTable, brand.toJson());
}

Future<int> updateBrand(Brand brand) async {
  final db = await database;
  return db.update(
    _brandTable,
    brand.toJson(),
    where: 'id = ?',
    whereArgs: [brand.id],
  );
}

// 2.4: Kiểm tra Brand có đang được sử dụng bởi Computer nào không
Future<int> countComputersByBrand(int brandId) async {
  final db = await database;
  final result = await db.rawQuery(
    'SELECT COUNT(*) AS cnt FROM $_computerTable WHERE brand_id = ?',
    [brandId],
```

```
    );
    return Sqflite.firstIntValue(result) ?? 0;
  }

  Future<int> deleteBrand(int id) async {
    final db = await database;
    return db.delete(_brandTable, where: 'id = ?', whereArgs: [id]);
  }

  // ==========================================================================
  // TASK 3: CATEGORY MANAGEMENT
  // ==========================================================================

  // 3.1 & 3.5: Lấy danh sách Category kèm số lượng sản phẩm (hỗ trợ tìm kiếm)
  Future<List<CategoryModel>> getCategories({String? keyword}) async {
    final db = await database;
    String whereClause = '';
    List<Object?> args = [];

    if (keyword != null && keyword.trim().isNotEmpty) {
      whereClause = 'WHERE ca.category_name LIKE ? COLLATE NOCASE';
      args = ['%${keyword.trim()}%'];
    }

    final maps = await db.rawQuery('''
      SELECT
        ca.id,
        ca.category_name,
        COUNT(c.id) AS totalProducts
      FROM $_categoryTable ca
      LEFT JOIN $_computerTable c ON ca.id = c.category_id
      $whereClause
      GROUP BY ca.id
      ORDER BY ca.category_name ASC
    ''', args);

    return [for (final map in maps) CategoryModel.fromJson(map)];
  }

  Future<bool> isCategoryNameDuplicate(String name, {int? excludeId}) async {
    final db = await database;
    final maps = await db.query(
      _categoryTable,
      where: excludeId != null
          ? 'LOWER(category_name) = LOWER(?) AND id != ?'
          : 'LOWER(category_name) = LOWER(?)',
      whereArgs: excludeId != null ? [name.trim(), excludeId] : [name.trim()],
```

```
    );
    return maps.isNotEmpty;
  }

  Future<int> insertCategory(CategoryModel category) async {
    final db = await database;
    return db.insert(_categoryTable, category.toJson());
  }

  Future<int> updateCategory(CategoryModel category) async {
    final db = await database;
    return db.update(
      _categoryTable,
      category.toJson(),
      where: 'id = ?',
      whereArgs: [category.id],
    );
  }

  // 3.4: Kiểm tra Category có đang được sử dụng không
  Future<int> countComputersByCategory(int categoryId) async {
    final db = await database;
    final result = await db.rawQuery(
      'SELECT COUNT(*) AS cnt FROM $_computerTable WHERE category_id = ?',
      [categoryId],
    );
    return Sqflite.firstIntValue(result) ?? 0;
  }

  Future<int> deleteCategory(int id) async {
    final db = await database;
    return db.delete(_categoryTable, where: 'id = ?', whereArgs: [id]);
  }

  // ============================================================================
  // TASK 4 & TASK 5: COMPUTER MANAGEMENT + SEARCH, FILTER, SORT
  // ============================================================================

  // 4.1 & 4.7: getAllComputers()
  Future<List<Computer>> getAllComputers() async {
    final db = await database;
    final maps = await db.rawQuery('''
      SELECT
        c.id,
        c.product_name,
        c.brand_id,
        c.category_id,
```

```
      b.brand_name,
      ca.category_name,
      c.cpu,
      c.ram,
      c.storage,
      c.price,
      c.stock,
      c.sold_quantity,
      c.is_featured
    FROM $_computerTable c
    INNER JOIN $_brandTable b ON c.brand_id = b.id
    INNER JOIN $_categoryTable ca ON c.category_id = ca.id
    ORDER BY c.product_name ASC
  ''');
  return [for (final map in maps) Computer.fromJson(map)];
}

// 4.5 & 4.7: getComputerById()
Future<Computer?> getComputerById(int id) async {
  final db = await database;
  final maps = await db.rawQuery('''
    SELECT
      c.*,
      b.brand_name,
      ca.category_name
    FROM $_computerTable c
    INNER JOIN $_brandTable b ON c.brand_id = b.id
    INNER JOIN $_categoryTable ca ON c.category_id = ca.id
    WHERE c.id = ?
  ''', [id]);

  if (maps.isEmpty) return null;
  return Computer.fromJson(maps.first);
}

Future<bool> isProductNameDuplicate(String name, {int? excludeId}) async {
  final db = await database;
  final maps = await db.query(
    _computerTable,
    where: excludeId != null
        ? 'LOWER(product_name) = LOWER(?) AND id != ?'
        : 'LOWER(product_name) = LOWER(?)',
    whereArgs: excludeId != null ? [name.trim(), excludeId] : [name.trim()],
  );
  return maps.isNotEmpty;
}
```

```
  // 4.2 & 4.7: insertComputer()
  Future<int> insertComputer(Computer computer) async {
    final db = await database;
    return db.rawInsert('''
      INSERT INTO $_computerTable (
        product_name, brand_id, category_id, cpu, ram, storage, price, stock,
sold_quantity, is_featured
      ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    ''', [
      computer.productName,
      computer.brandId,
      computer.categoryId,
      computer.cpu,
      computer.ram,
      computer.storage,
      computer.price,
      computer.stock,
      computer.soldQuantity,
      computer.isFeatured ? 1 : 0,
    ]);
  }

  // 4.3 & 4.7: updateComputer()
  Future<int> updateComputer(Computer computer) async {
    final db = await database;
    return db.rawUpdate('''
      UPDATE $_computerTable
      SET
        product_name = ?,
        brand_id = ?,
        category_id = ?,
        cpu = ?,
        ram = ?,
        storage = ?,
        price = ?,
        stock = ?,
        sold_quantity = ?,
        is_featured = ?
      WHERE id = ?
    ''', [
      computer.productName,
      computer.brandId,
      computer.categoryId,
      computer.cpu,
      computer.ram,
      computer.storage,
```

```
    computer.price,
    computer.stock,
    computer.soldQuantity,
    computer.isFeatured ? 1 : 0,
    computer.id,
  ]);
}

// 4.4 & 4.7: deleteComputer()
Future<int> deleteComputer(int id) async {
  final db = await database;
  return db.rawDelete('DELETE FROM $_computerTable WHERE id = ?', [id]);
}

// 5.1 & 5.8: searchComputers()
Future<List<Computer>> searchComputers(String keyword) async {
  return queryComputersDynamic(keyword: keyword);
}

// 5.2, 5.3, 5.6 & 5.8: filterComputers() & sortComputers() kết hợp động
Future<List<Computer>> filterComputers({
  String? keyword,
  int? brandId,
  int? categoryId,
  bool? featuredOnly,
  String? stockStatus, // 'in_stock', 'low_stock', 'out_of_stock'
  String? ramOption,   // '8', '16', '32+'
  String? storageOption, // '256', '512', '1024', '2048+'
  String? priceOption, // '<500', '500-1000', '>1000'
  String? sortBy,
}) async {
  return queryComputersDynamic(
    keyword: keyword,
    brandId: brandId,
    categoryId: categoryId,
    featuredOnly: featuredOnly,
    stockStatus: stockStatus,
    ramOption: ramOption,
    storageOption: storageOption,
    priceOption: priceOption,
    sortBy: sortBy,
  );
}

Future<List<Computer>> sortComputers(String sortBy) async {
  return queryComputersDynamic(sortBy: sortBy);
}
```

```
  Future<List<Computer>> resetFilters() async {
    return getAllComputers();
  }

  // Hàm xây dựng câu lệnh SQL động (Dynamic SQL Builder) cho Task 5
  Future<List<Computer>> queryComputersDynamic({
    String? keyword,
    int? brandId,
    int? categoryId,
    bool? featuredOnly,
    String? stockStatus,
    String? ramOption,
    String? storageOption,
    String? priceOption,
    String? sortBy,
  }) async {
    final db = await database;
    final List<String> conditions = [];
    final List<Object?> args = [];

    // 5.1 Search theo Product Name, Brand Name, hoặc CPU (không phân biệt
hoa/thường)
    if (keyword != null && keyword.trim().isNotEmpty) {
      final kw = '%${keyword.trim()}%';
      conditions.add(
        '(c.product_name LIKE ? COLLATE NOCASE OR b.brand_name LIKE ? COLLATE
NOCASE OR c.cpu LIKE ? COLLATE NOCASE)',
      );
      args.addAll([kw, kw, kw]);
    }

    // 5.2 Filter by Brand
    if (brandId != null) {
      conditions.add('b.id = ?');
      args.add(brandId);
    }

    // 5.2 Filter by Category
    if (categoryId != null) {
      conditions.add('ca.id = ?');
      args.add(categoryId);
    }

    // 5.2 Filter by Featured
    if (featuredOnly == true) {
      conditions.add('c.is_featured = 1');
```

```
}

// 5.2 Filter by Stock Status
if (stockStatus == 'in_stock') {
  conditions.add('c.stock > 0');
} else if (stockStatus == 'low_stock') {
  conditions.add('c.stock < 5');
} else if (stockStatus == 'out_of_stock') {
  conditions.add('c.stock = 0');
}

// 5.2 Filter by RAM
if (ramOption == '8') {
  conditions.add('c.ram = 8');
} else if (ramOption == '16') {
  conditions.add('c.ram = 16');
} else if (ramOption == '32+') {
  conditions.add('c.ram >= 32');
}

// 5.2 Filter by Storage
if (storageOption == '256') {
  conditions.add('c.storage = 256');
} else if (storageOption == '512') {
  conditions.add('c.storage = 512');
} else if (storageOption == '1024') {
  conditions.add('c.storage = 1024');
} else if (storageOption == '2048+') {
  conditions.add('c.storage >= 2048');
}

// 5.2 Filter by Price
if (priceOption == '<500') {
  conditions.add('c.price < 500');
} else if (priceOption == '500-1000') {
  conditions.add('c.price BETWEEN 500 AND 1000');
} else if (priceOption == '>1000') {
  conditions.add('c.price > 1000');
}

final String whereSql =
    conditions.isEmpty ? '' : 'WHERE ${conditions.join(' AND ')}';

// 5.4 Sort Options
String orderSql = 'ORDER BY c.product_name ASC';
switch (sortBy) {
  case 'price_asc':
```

```
        orderSql = 'ORDER BY c.price ASC';
        break;
      case 'price_desc':
        orderSql = 'ORDER BY c.price DESC';
        break;
      case 'sold_asc':
        orderSql = 'ORDER BY c.sold_quantity ASC';
        break;
      case 'sold_desc':
        orderSql = 'ORDER BY c.sold_quantity DESC';
        break;
      case 'name_asc':
        orderSql = 'ORDER BY c.product_name ASC';
        break;
      case 'name_desc':
        orderSql = 'ORDER BY c.product_name DESC';
        break;
      case 'brand_asc':
        orderSql = 'ORDER BY b.brand_name ASC';
        break;
      case 'brand_desc':
        orderSql = 'ORDER BY b.brand_name DESC';
        break;
      case 'stock_asc':
        orderSql = 'ORDER BY c.stock ASC';
        break;
      case 'stock_desc':
        orderSql = 'ORDER BY c.stock DESC';
        break;
    }

    final maps = await db.rawQuery('''
      SELECT
        c.id,
        c.product_name,
        c.brand_id,
        c.category_id,
        b.brand_name,
        ca.category_name,
        c.cpu,
        c.ram,
        c.storage,
        c.price,
        c.stock,
        c.sold_quantity,
        c.is_featured
```

```
      FROM $_computerTable c
      INNER JOIN $_brandTable b ON c.brand_id = b.id
      INNER JOIN $_categoryTable ca ON c.category_id = ca.id
      $whereSql
      $orderSql
    ''', args);

    return [for (final map in maps) Computer.fromJson(map)];
  }

  // ============================================================================
  // TASK 6: STATISTICS SCREEN METHODS (SQLite Aggregate Functions)
  // ============================================================================

  // 6.1 & 6.9: getStatistics()
  Future<GeneralStats> getStatistics() async {
    final db = await database;

    final countRes = await db.rawQuery('SELECT COUNT(*) AS val FROM
$_computerTable');
    final revRes = await db.rawQuery('SELECT SUM(price * sold_quantity) AS val FROM
$_computerTable');
    final soldRes = await db.rawQuery('SELECT SUM(sold_quantity) AS val FROM
$_computerTable');
    final avgRes = await db.rawQuery('SELECT AVG(price) AS val FROM
$_computerTable');
    final invRes = await db.rawQuery('SELECT SUM(price * stock) AS val FROM
$_computerTable');

    return GeneralStats(
      totalProducts: (countRes.first['val'] as num?)?.toInt() ?? 0,
      totalRevenue: (revRes.first['val'] as num?)?.toDouble() ?? 0.0,
      totalSoldQuantity: (soldRes.first['val'] as num?)?.toInt() ?? 0,
      averagePrice: (avgRes.first['val'] as num?)?.toDouble() ?? 0.0,
      totalInventoryValue: (invRes.first['val'] as num?)?.toDouble() ?? 0.0,
    );
  }

  // 6.2 & 6.9: getBrandStatistics()
  Future<List<BrandStatItem>> getBrandStatistics() async {
    final db = await database;
    final maps = await db.rawQuery('''
      SELECT
        b.brand_name,
        COUNT(c.id) AS TotalProducts,
        COALESCE(SUM(c.sold_quantity), 0) AS TotalSold,
        COALESCE(SUM(c.price * c.sold_quantity), 0) AS Revenue,
```

```
      COALESCE(SUM(c.price * c.stock), 0) AS InventoryValue
    FROM $_brandTable b
    LEFT JOIN $_computerTable c ON b.id = c.brand_id
    GROUP BY b.id
    ORDER BY Revenue DESC
  ''');
  return [for (final m in maps) BrandStatItem.fromJson(m)];
}

// 6.3 & 6.9: getCategoryStatistics()
Future<List<CategoryStatItem>> getCategoryStatistics() async {
  final db = await database;
  final maps = await db.rawQuery('''
    SELECT
      ca.category_name,
      COUNT(c.id) AS totalProducts,
      COALESCE(SUM(c.price * c.sold_quantity), 0) AS totalRevenue,
      COALESCE(SUM(c.sold_quantity), 0) AS totalSold
    FROM $_categoryTable ca
    LEFT JOIN $_computerTable c ON ca.id = c.category_id
    GROUP BY ca.id
  ''');
  return [for (final m in maps) CategoryStatItem.fromJson(m)];
}

// 6.4 & 6.9: getBestSellingProduct()
Future<Map<String, dynamic>?> getBestSellingProduct() async {
  final db = await database;
  final maps = await db.rawQuery('''
    SELECT product_name, sold_quantity
    FROM $_computerTable
    ORDER BY sold_quantity DESC
    LIMIT 1
  ''');
  return maps.isEmpty ? null : maps.first;
}

// 6.5: getMostExpensiveProduct()
Future<Map<String, dynamic>?> getMostExpensiveProduct() async {
  final db = await database;
  final maps = await db.rawQuery('''
    SELECT product_name, price
    FROM $_computerTable
    ORDER BY price DESC
    LIMIT 1
  ''');
```

```
  return maps.isEmpty ? null : maps.first;
}

// 6.6 & 6.9: getLowStockProducts()
Future<List<Map<String, dynamic>>> getLowStockProducts() async {
  final db = await database;
  return db.rawQuery('''
    SELECT
      c.product_name,
      b.brand_name,
      c.stock
    FROM $_computerTable c
    INNER JOIN $_brandTable b ON c.brand_id = b.id
    WHERE c.stock < 5
  ''');
}

// 6.7 & 6.9: getTopBrand()
Future<Map<String, dynamic>?> getTopBrand() async {
  final db = await database;
  final maps = await db.rawQuery('''
    SELECT
      b.brand_name,
      SUM(c.sold_quantity) AS TotalSold
    FROM $_brandTable b
    INNER JOIN $_computerTable c ON b.id = c.brand_id
    GROUP BY b.id
    ORDER BY TotalSold DESC
    LIMIT 1
  ''');
  return maps.isEmpty ? null : maps.first;
}

// 6.8: getFeaturedStatistics()
Future<FeaturedStats> getFeaturedStatistics() async {
  final db = await database;
  final maps = await db.rawQuery('''
    SELECT
      COUNT(*) AS totalFeatured,
      COALESCE(SUM(price * sold_quantity), 0) AS featuredRevenue,
      COALESCE(SUM(sold_quantity), 0) AS featuredSold
    FROM $_computerTable
    WHERE is_featured = 1
  ''');
  final row = maps.first;
  return FeaturedStats(
```

```
      totalFeaturedProducts: (row['totalFeatured'] as num?)?.toInt() ?? 0,
      totalFeaturedRevenue: (row['featuredRevenue'] as num?)?.toDouble() ?? 0.0,
      totalFeaturedSoldQuantity: (row['featuredSold'] as num?)?.toInt() ?? 0,
    );
  }
}
```

Bước 4: Xây dựng tầng `lib/providers/` với Riverpod (Theo đúng tư duy `note_notifier.dart`)

Giải thích

Trong , ta sử dụng `@riverpod` class kế thừa `_$...Notifier`. Khi có thay đổi ở bảng `computers` (thêm/sửa/xóa), ngoài việc làm mới `computerProvider`, ta cũng gọi `ref.invalidate(brandProvider)`, `ref.invalidate(categoryProvider)` và `ref.invalidate(statisticsProvider)` để số lượng sản phẩm ở tab Brand, Category và số liệu ở tab Statistics tự động cập nhật ngay lập tức.

4.1. File `lib/providers/brand_notifier.dart`

```
dart
import 'package:riverpod_annotation/riverpod_annotation.dart';
import '../db/database_helper.dart';
import '../model/brand.dart';

part 'brand_notifier.g.dart';

@riverpod
class BrandNotifier extends _$BrandNotifier {
  String _searchKeyword = '';

  @override
  Future<List<Brand>> build() async {
    return DatabaseHelper.instance.getBrands(keyword: _searchKeyword);
  }

  Future<void> searchBrands(String keyword) async {
    _searchKeyword = keyword;
    ref.invalidateSelf();
    await future;
  }

  Future<void> addBrand(Brand brand) async {
    await DatabaseHelper.instance.insertBrand(brand);
    ref.invalidateSelf();
    await future;
  }

  Future<void> updateBrand(Brand brand) async {
    await DatabaseHelper.instance.updateBrand(brand);
    ref.invalidateSelf();
    await future;
  }

  // Trả về true nếu xóa thành công, false nếu đang được sử dụng bởi Computer
  Future<bool> deleteBrand(int id) async {
    final usedCount = await DatabaseHelper.instance.countComputersByBrand(id);
    if (usedCount > 0) {
      return false;
    }
    await DatabaseHelper.instance.deleteBrand(id);
    ref.invalidateSelf();
    await future;
    return true;
```

```
  }
}
```

4.2. File `lib/providers/category_notifier.dart`

```
dart
import 'package:riverpod_annotation/riverpod_annotation.dart';
import '../db/database_helper.dart';
import '../model/category.dart';

part 'category_notifier.g.dart';

@riverpod
class CategoryNotifier extends _$CategoryNotifier {
  String _searchKeyword = '';

  @override
  Future<List<CategoryModel>> build() async {
    return DatabaseHelper.instance.getCategories(keyword: _searchKeyword);
  }

  Future<void> searchCategories(String keyword) async {
    _searchKeyword = keyword;
    ref.invalidateSelf();
    await future;
  }

  Future<void> addCategory(CategoryModel category) async {
    await DatabaseHelper.instance.insertCategory(category);
    ref.invalidateSelf();
    await future;
  }

  Future<void> updateCategory(CategoryModel category) async {
    await DatabaseHelper.instance.updateCategory(category);
    ref.invalidateSelf();
    await future;
  }

  Future<bool> deleteCategory(int id) async {
    final usedCount = await DatabaseHelper.instance.countComputersByCategory(id);
    if (usedCount > 0) {
      return false;
    }
    await DatabaseHelper.instance.deleteCategory(id);
    ref.invalidateSelf();
    await future;
    return true;
  }
```

```
}
```

4.3. File `lib/providers/computer_notifier.dart`

```
dart
import 'package:riverpod_annotation/riverpod_annotation.dart';
import '../db/database_helper.dart';
import '../model/computer.dart';
import 'brand_notifier.dart';
import 'category_notifier.dart';
import 'statistics_notifier.dart';

part 'computer_notifier.g.dart';

@riverpod
class ComputerNotifier extends _$ComputerNotifier {
  // Các trạng thái Search, Filter, Sort hiện tại của màn hình Store
  String keyword = '';
  int? selectedBrandId;
  int? selectedCategoryId;
  bool featuredOnly = false;
  String? stockStatus;   // 'in_stock', 'low_stock', 'out_of_stock'
  String? ramOption;     // '8', '16', '32+'
  String? storageOption; // '256', '512', '1024', '2048+'
  String? priceOption;   // '<500', '500-1000', '>1000'
  String? sortBy;        // 'price_asc', 'price_desc', ...

  @override
  Future<List<Computer>> build() async {
    return DatabaseHelper.instance.filterComputers(
      keyword: keyword,
      brandId: selectedBrandId,
      categoryId: selectedCategoryId,
      featuredOnly: featuredOnly,
      stockStatus: stockStatus,
      ramOption: ramOption,
      storageOption: storageOption,
      priceOption: priceOption,
      sortBy: sortBy,
    );
  }

  void _invalidateRelatedProviders() {
    ref.invalidate(brandProvider);
    ref.invalidate(categoryProvider);
    ref.invalidate(statisticsProvider);
  }
```

```
Future<void> addComputer(Computer computer) async {
  await DatabaseHelper.instance.insertComputer(computer);
  _invalidateRelatedProviders();
  ref.invalidateSelf();
  await future;
}

Future<void> updateComputer(Computer computer) async {
  await DatabaseHelper.instance.updateComputer(computer);
  _invalidateRelatedProviders();
  ref.invalidateSelf();
  await future;
}

Future<void> deleteComputer(int id) async {
  await DatabaseHelper.instance.deleteComputer(id);
  _invalidateRelatedProviders();
  ref.invalidateSelf();
  await future;
}

Future<void> setSearchKeyword(String value) async {
  keyword = value;
  ref.invalidateSelf();
  await future;
}

Future<void> applyFilters({
  int? brandId,
  int? categoryId,
  bool? featured,
  String? stock,
  String? ram,
  String? storage,
  String? price,
}) async {
  selectedBrandId = brandId;
  selectedCategoryId = categoryId;
  featuredOnly = featured ?? false;
  stockStatus = stock;
  ramOption = ram;
  storageOption = storage;
  priceOption = price;
  ref.invalidateSelf();
  await future;
}
```

```
  Future<void> setSortOption(String? sortOption) async {
    sortBy = sortOption;
    ref.invalidateSelf();
    await future;
  }

  Future<void> resetAllFilters() async {
    keyword = '';
    selectedBrandId = null;
    selectedCategoryId = null;
    featuredOnly = false;
    stockStatus = null;
    ramOption = null;
    storageOption = null;
    priceOption = null;
    sortBy = null;
    await DatabaseHelper.instance.resetFilters();
    ref.invalidateSelf();
    await future;
  }
}
```

4.4. File `lib/providers/statistics_notifier.dart`

```
dart
import 'package:riverpod_annotation/riverpod_annotation.dart';
import '../db/database_helper.dart';
import '../model/store_statistics.dart';

part 'statistics_notifier.g.dart';

@riverpod
class StatisticsNotifier extends _$StatisticsNotifier {
  @override
  Future<DashboardStatistics> build() async {
    final db = DatabaseHelper.instance;
    final general = await db.getStatistics();
    final byBrand = await db.getBrandStatistics();
    final byCategory = await db.getCategoryStatistics();
    final bestSelling = await db.getBestSellingProduct();
    final mostExpensive = await db.getMostExpensiveProduct();
    final lowStock = await db.getLowStockProducts();
    final topBrand = await db.getTopBrand();
    final featuredStats = await db.getFeaturedStatistics();

    return DashboardStatistics(
      general: general,
      byBrand: byBrand,
      byCategory: byCategory,
      bestSellingProduct: bestSelling,
      mostExpensiveProduct: mostExpensive,
      lowStockProducts: lowStock,
      topBrand: topBrand,
      featuredStats: featuredStats,
    );
  }
}
```

> TIP
>
> Sau khi tạo xong 4 file trong `lib/providers/`, chạy lệnh sau trong Terminal để `riverpod_generator` tự động sinh ra 4 file `.g.dart` giống như `note_notifier.g.dart` của `sqlite_intro`:
>
> ```
> bash
> dart run build_runner build --delete-conflicting-outputs
> ```

Bước 5: Xây dựng Tầng Giao Diện `lib/ui/` (6 Tabs & Các Màn Hình Chi Tiết)

5.1. Màn hình điều hướng chính `lib/ui/main_navigation_screen.dart` (Navigation Structure)

- Sử dụng `BottomNavigationBar` (type: `BottomNavigationBarType.fixed`) luôn hiển thị cố định với **6 tabs**:
    1. `Store` (mặc định mở đầu tiên `currentIndex = 0`)
    2. `Add Computer`
    3. `Brands`
    4. `Categories`
    5. `Statistics`
    6. `Featured`

```dart
import 'package:flutter/material.dart';
import 'store_screen.dart';
import 'computer_form_screen.dart';
import 'brand_list_screen.dart';
import 'category_list_screen.dart';
import 'statistics_screen.dart';
import 'featured_screen.dart';

class MainNavigationScreen extends StatefulWidget {
  const MainNavigationScreen({super.key});

  @override
  State<MainNavigationScreen> createState() => _MainNavigationScreenState();
}

class _MainNavigationScreenState extends State<MainNavigationScreen> {
  int _currentIndex = 0;

  void _switchToStoreTab() {
    setState(() {
      _currentIndex = 0;
    });
  }

  @override
  Widget build(BuildContext context) {
    final screens = [
      const StoreScreen(),
      ComputerFormScreen(onSavedSuccess: _switchToStoreTab),
      const BrandListScreen(),
      const CategoryListScreen(),
      const StatisticsScreen(),
      const FeaturedScreen(),
    ];

    return Scaffold(
      body: screens[_currentIndex],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _currentIndex,
        type: BottomNavigationBarType.fixed,
        selectedItemColor: Colors.indigo,
        unselectedItemColor: Colors.grey,
        onTap: (index) {
          setState(() {
            _currentIndex = index;
```

```
          });
        },
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.store), label: 'Store'),
          BottomNavigationBarItem(icon: Icon(Icons.add_box), label: 'Add
Computer'),
          BottomNavigationBarItem(icon: Icon(Icons.business), label: 'Brands'),
          BottomNavigationBarItem(icon: Icon(Icons.category), label: 'Categories'),
          BottomNavigationBarItem(icon: Icon(Icons.bar_chart), label:
'Statistics'),
          BottomNavigationBarItem(icon: Icon(Icons.star), label: 'Featured'),
        ],
      ),
    );
  }
}
```

5.2. Tab 3 — Quản lý Brand `lib/ui/brand_list_screen.dart` (Task 2)

Đáp ứng toàn bộ yêu cầu Task 2.1 → 2.5:

- Hiển thị danh sách bằng `ListView.builder()` + `Card`.
- Hiển thị `Brand Name`, `Country`, `Founded Year`, `Products: <totalProducts>` (tính từ `LEFT JOIN`).
- Thanh tìm kiếm thời gian thực (Real-time search) theo `Brand Name` hoặc `Country`.
- Form Thêm / Sửa Brand kiểm tra đầy đủ:
  - `Brand Name` không trống & không được trùng (`isBrandNameDuplicate`).
  - `Country` không trống.
  - `Founded Year > 1900`.
  - Hiện Snackbar `"Brand added successfully."` và tự động làm mới danh sách.
- Xóa Brand:
  - Hiển thị `AlertDialog`: `"Are you sure you want to delete this brand?"`.
  - Nếu `count > 0` hiển thị đúng thông báo: `"This brand cannot be deleted because it is currently used by one or more products."`.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../db/database_helper.dart';
import '../model/brand.dart';
import '../providers/brand_notifier.dart';

class BrandListScreen extends ConsumerStatefulWidget {
  const BrandListScreen({super.key});

  @override
  ConsumerState<BrandListScreen> createState() => _BrandListScreenState();
}

class _BrandListScreenState extends ConsumerState<BrandListScreen> {
  final TextEditingController _searchCtrl = TextEditingController();

  @override
  void dispose() {
    _searchCtrl.dispose();
    super.dispose();
  }

  Future<void> _showBrandDialog({Brand? existingBrand}) async {
    final formKey = GlobalKey<FormState>();
    final nameCtrl = TextEditingController(text: existingBrand?.brandName ?? '');
    final countryCtrl = TextEditingController(text: existingBrand?.country ?? '');
    final yearCtrl = TextEditingController(
      text: existingBrand != null ? existingBrand.foundedYear.toString() : '',
    );
    String? duplicateError;

    await showDialog(
      context: context,
      builder: (ctx) {
        return StatefulBuilder(
          builder: (ctx, setDialogState) {
            return AlertDialog(
              title: Text(existingBrand == null ? 'Add Brand' : 'Edit Brand'),
              content: Form(
                key: formKey,
                child: SingleChildScrollView(
                  child: Column(
                    mainAxisSize: MainAxisSize.min,
                    children: [
                      TextFormField(
```

```
                        controller: nameCtrl,
                        decoration: InputDecoration(
                          labelText: 'Brand Name',
                          errorText: duplicateError,
                        ),
                        validator: (val) {
                          if (val == null || val.trim().isEmpty) {
                            return 'Brand Name cannot be empty.';
                          }
                          return null;
                        },
                      ),
                      const SizedBox(height: 12),
                      TextFormField(
                        controller: countryCtrl,
                        decoration: const InputDecoration(labelText: 'Country'),
                        validator: (val) {
                          if (val == null || val.trim().isEmpty) {
                            return 'Country cannot be empty.';
                          }
                          return null;
                        },
                      ),
                      const SizedBox(height: 12),
                      TextFormField(
                        controller: yearCtrl,
                        keyboardType: TextInputType.number,
                        decoration: const InputDecoration(labelText: 'Founded
Year'),
                        validator: (val) {
                          if (val == null || val.trim().isEmpty) {
                            return 'Founded Year is required.';
                          }
                          final year = int.tryParse(val.trim());
                          if (year == null || year <= 1900) {
                            return 'Founded Year must be greater than 1900.';
                          }
                          return null;
                        },
                      ),
                    ],
                  ),
                ),
              ),
              actions: [
                TextButton(
```

```
                  onPressed: () => Navigator.pop(ctx),
                  child: const Text('Cancel'),
                ),
                ElevatedButton(
                  onPressed: () async {
                    setDialogState(() => duplicateError = null);
                    if (!formKey.currentState!.validate()) return;

                    final isDup = await
DatabaseHelper.instance.isBrandNameDuplicate(
                      nameCtrl.text.trim(),
                      excludeId: existingBrand?.id,
                    );
                    if (isDup) {
                      setDialogState(() {
                        duplicateError = 'Duplicate Brand Name is not allowed.';
                      });
                      return;
                    }

                    final brand = Brand(
                      id: existingBrand?.id,
                      brandName: nameCtrl.text.trim(),
                      country: countryCtrl.text.trim(),
                      foundedYear: int.parse(yearCtrl.text.trim()),
                    );

                    if (existingBrand == null) {
                      await ref.read(brandProvider.notifier).addBrand(brand);
                      if (mounted) {
                        Navigator.pop(ctx);
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(content: Text('Brand added
successfully.')),
                        );
                      }
                    } else {
                      await ref.read(brandProvider.notifier).updateBrand(brand);
                      if (mounted) {
                        Navigator.pop(ctx);
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(content: Text('Brand updated
successfully.')),
                        );
                      }
                    }
```

```
                  },
                  child: const Text('Save'),
                ),
              ],
            );
          },
        );
      },
    );
  }

  Future<void> _confirmDeleteBrand(Brand brand) async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Delete Brand'),
        content: const Text('Are you sure you want to delete this brand?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: const Text('Cancel'),
          ),
          ElevatedButton(
            style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
            onPressed: () => Navigator.pop(ctx, true),
            child: const Text('Delete', style: TextStyle(color: Colors.white)),
          ),
        ],
      ),
    );

    if (confirmed == true) {
      final success = await
ref.read(brandProvider.notifier).deleteBrand(brand.id!);
      if (!mounted) return;
      if (!success) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            backgroundColor: Colors.red,
            content: Text(
              'This brand cannot be deleted because it is currently used by one or
more products.',
            ),
          ),
        );
      } else {
```

```
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Delete successfully.')),
        );
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    final brandsAsync = ref.watch(brandProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Brand Management')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12.0),
            child: TextField(
              controller: _searchCtrl,
              decoration: const InputDecoration(
                hintText: 'Search by Brand Name or Country...',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
              ),
              onChanged: (val) {
                ref.read(brandProvider.notifier).searchBrands(val);
              },
            ),
          ),
          Expanded(
            child: brandsAsync.when(
              data: (brands) => brands.isEmpty
                  ? const Center(child: Text('No brands found.'))
                  : ListView.builder(
                      itemCount: brands.length,
                      itemBuilder: (context, index) {
                        final brand = brands[index];
                        return Card(
                          margin: const EdgeInsets.symmetric(horizontal: 12,
vertical: 6),
                          child: ListTile(
                            title: Text(
                              brand.brandName,
                              style: const TextStyle(fontWeight: FontWeight.bold,
fontSize: 16),
                            ),
```

```
                            subtitle: Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
                                const SizedBox(height: 4),
                                Text('Country : ${brand.country}'),
                                Text('Founded : ${brand.foundedYear}'),
                                Text('Products : ${brand.totalProducts}'),
                              ],
                            ),
                            trailing: Row(
                              mainAxisSize: MainAxisSize.min,
                              children: [
                                IconButton(
                                  icon: const Icon(Icons.edit, color: Colors.blue),
                                  onPressed: () => _showBrandDialog(existingBrand:
brand),
                                ),
                                IconButton(
                                  icon: const Icon(Icons.delete, color:
Colors.red),
                                  onPressed: () => _confirmDeleteBrand(brand),
                                ),
                              ],
                            ),
                          ),
                        );
                      },
                    ),
              error: (err, stack) => Center(child: Text('Error: $err')),
              loading: () => const Center(child: CircularProgressIndicator()),
            ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _showBrandDialog(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

5.3. Tab 4 — Quản lý Category `lib/ui/category_list_screen.dart` (Task 3)

Đáp ứng toàn bộ yêu cầu Task 3.1 → 3.5:

- Hiển thị `Category Name` và `Products : <totalProducts>` tính từ `LEFT JOIN`.
- Real-time search theo `Category Name`.
- Thêm/Sửa Category với kiểm tra không trống, không trùng tên; hiển thị `"Category added successfully."`.
- Xóa Category: hiển thị `"Are you sure you want to delete this category?"`, kiểm tra `COUNT(*)`, nếu đang được sử dụng báo `"This category cannot be deleted because one or more products belong to this category."`.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../db/database_helper.dart';
import '../model/category.dart';
import '../providers/category_notifier.dart';

class CategoryListScreen extends ConsumerStatefulWidget {
  const CategoryListScreen({super.key});

  @override
  ConsumerState<CategoryListScreen> createState() => _CategoryListScreenState();
}

class _CategoryListScreenState extends ConsumerState<CategoryListScreen> {
  final TextEditingController _searchCtrl = TextEditingController();

  @override
  void dispose() {
    _searchCtrl.dispose();
    super.dispose();
  }

  Future<void> _showCategoryDialog({CategoryModel? existingCategory}) async {
    final formKey = GlobalKey<FormState>();
    final nameCtrl = TextEditingController(text: existingCategory?.categoryName ??
'');
    String? duplicateError;

    await showDialog(
      context: context,
      builder: (ctx) {
        return StatefulBuilder(
          builder: (ctx, setDialogState) {
            return AlertDialog(
              title: Text(existingCategory == null ? 'Add Category' : 'Edit
Category'),
              content: Form(
                key: formKey,
                child: TextFormField(
                  controller: nameCtrl,
                  decoration: InputDecoration(
                    labelText: 'Category Name',
                    errorText: duplicateError,
                  ),
                  validator: (val) {
```

```
                    if (val == null || val.trim().isEmpty) {
                      return 'Category Name is required.';
                    }
                    return null;
                  },
                ),
              ),
              actions: [
                TextButton(
                  onPressed: () => Navigator.pop(ctx),
                  child: const Text('Cancel'),
                ),
                ElevatedButton(
                  onPressed: () async {
                    setDialogState(() => duplicateError = null);
                    if (!formKey.currentState!.validate()) return;

                    final isDup = await
DatabaseHelper.instance.isCategoryNameDuplicate(
                      nameCtrl.text.trim(),
                      excludeId: existingCategory?.id,
                    );
                    if (isDup) {
                      setDialogState(() {
                        duplicateError = 'Category Name must be unique.';
                      });
                      return;
                    }

                    final category = CategoryModel(
                      id: existingCategory?.id,
                      categoryName: nameCtrl.text.trim(),
                    );

                    if (existingCategory == null) {
                      await
ref.read(categoryProvider.notifier).addCategory(category);
                      if (mounted) {
                        Navigator.pop(ctx);
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(content: Text('Category added
successfully.')),
                        );
                      }
                    } else {
```

```
                      await
ref.read(categoryProvider.notifier).updateCategory(category);
                      if (mounted) {
                        Navigator.pop(ctx);
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(content: Text('Category updated
successfully.')),
                        );
                      }
                    }
                  },
                  child: const Text('Save'),
                ),
              ],
            );
          },
        );
      },
    );
  }

  Future<void> _confirmDeleteCategory(CategoryModel category) async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Delete Category'),
        content: const Text('Are you sure you want to delete this category?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: const Text('Cancel'),
          ),
          ElevatedButton(
            style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
            onPressed: () => Navigator.pop(ctx, true),
            child: const Text('Delete', style: TextStyle(color: Colors.white)),
          ),
        ],
      ),
    );

    if (confirmed == true) {
      final success = await
ref.read(categoryProvider.notifier).deleteCategory(category.id!);
      if (!mounted) return;
      if (!success) {
```

```
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
            backgroundColor: Colors.red,
            content: Text(
              'This category cannot be deleted because one or more products belong
to this category.',
            ),
          ),
        );
      } else {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Delete successfully.')),
        );
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    final categoriesAsync = ref.watch(categoryProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Category Management')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12.0),
            child: TextField(
              controller: _searchCtrl,
              decoration: const InputDecoration(
                hintText: 'Search by Category Name...',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
              ),
              onChanged: (val) {
                ref.read(categoryProvider.notifier).searchCategories(val);
              },
            ),
          ),
          Expanded(
            child: categoriesAsync.when(
              data: (categories) => categories.isEmpty
                  ? const Center(child: Text('No categories found.'))
                  : ListView.builder(
                      itemCount: categories.length,
                      itemBuilder: (context, index) {
```

```
                        final cat = categories[index];
                        return Card(
                          margin: const EdgeInsets.symmetric(horizontal: 12,
vertical: 6),
                          child: ListTile(
                            title: Text(
                              cat.categoryName,
                              style: const TextStyle(fontWeight: FontWeight.bold,
fontSize: 16),
                            ),
                            subtitle: Text('Products : ${cat.totalProducts}'),
                            trailing: Row(
                              mainAxisSize: MainAxisSize.min,
                              children: [
                                IconButton(
                                  icon: const Icon(Icons.edit, color: Colors.blue),
                                  onPressed: () =>
_showCategoryDialog(existingCategory: cat),
                                ),
                                IconButton(
                                  icon: const Icon(Icons.delete, color:
Colors.red),
                                  onPressed: () => _confirmDeleteCategory(cat),
                                ),
                              ],
                            ),
                          ),
                        );
                      },
                    ),
              error: (err, stack) => Center(child: Text('Error: $err')),
              loading: () => const Center(child: CircularProgressIndicator()),
            ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _showCategoryDialog(),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

5.4. Tab 1 — Store Screen (Home Screen) `lib/ui/store_screen.dart` (Task 2.6, 4.1, 5.1 → 5.7)

Đảm bảo đúng thứ tự các thành phần trên màn hình chính theo trang 2 của đề bài:

1. `AppBar`
2. `Promotional Banner` (load từ `assets/images/promo_banner.png`, full width, bo góc, giữ tỉ lệ)
3. `Search Bar` (Real-time search theo Product Name, Brand Name, CPU)
4. `Filter Button` + `Sort Button` + `Reset Filters Button`
5. `Computer Product List` (Cuộn độc lập trong `Expanded` + `RefreshIndicator` cho Pull-to-Refresh, dùng `ListView.builder()`, bấm vào sản phẩm chuyển tới `ComputerDetailScreen` bằng `Navigator.push()`).

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/computer.dart';
import '../providers/brand_notifier.dart';
import '../providers/category_notifier.dart';
import '../providers/computer_notifier.dart';
import 'computer_detail_screen.dart';

class StoreScreen extends ConsumerStatefulWidget {
  const StoreScreen({super.key});

  @override
  ConsumerState<StoreScreen> createState() => _StoreScreenState();
}

class _StoreScreenState extends ConsumerState<StoreScreen> {
  final TextEditingController _searchCtrl = TextEditingController();

  @override
  void initState() {
    super.initState();
    _searchCtrl.text = ref.read(computerProvider.notifier).keyword;
  }

  @override
  void dispose() {
    _searchCtrl.dispose();
    super.dispose();
  }

  // Modal chọn nhiều điều kiện lọc (Multiple Filters - Task 5.2, 5.3, 5.6)
  Future<void> _openFilterSheet() async {
    final notifier = ref.read(computerProvider.notifier);
    final brands = await ref.read(brandProvider.future);
    final categories = await ref.read(categoryProvider.future);

    int? tempBrandId = notifier.selectedBrandId;
    int? tempCategoryId = notifier.selectedCategoryId;
    bool tempFeatured = notifier.featuredOnly;
    String? tempStock = notifier.stockStatus;
    String? tempRam = notifier.ramOption;
    String? tempStorage = notifier.storageOption;
    String? tempPrice = notifier.priceOption;

    if (!mounted) return;
```

```
    await showModalBottomSheet(
      context: context,
      isScrollControlled: true,
      builder: (ctx) {
        return StatefulBuilder(
          builder: (ctx, setSheetState) {
            return Padding(
              padding: EdgeInsets.only(
                left: 16,
                right: 16,
                top: 16,
                bottom: MediaQuery.of(ctx).viewInsets.bottom + 16,
              ),
              child: SingleChildScrollView(
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const Text(
                      'Filter Computers',
                      style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                    ),
                    const SizedBox(height: 12),
                    DropdownButtonFormField<int?>(
                      value: tempBrandId,
                      decoration: const InputDecoration(labelText: 'Brand'),
                      items: [
                        const DropdownMenuItem(value: null, child: Text('All
Brands')),
                        ...brands.map(
                          (b) => DropdownMenuItem(value: b.id, child:
Text(b.brandName)),
                        ),
                      ],
                      onChanged: (val) => setSheetState(() => tempBrandId = val),
                    ),
                    const SizedBox(height: 8),
                    DropdownButtonFormField<int?>(
                      value: tempCategoryId,
                      decoration: const InputDecoration(labelText: 'Category'),
                      items: [
                        const DropdownMenuItem(value: null, child: Text('All
Categories')),
                        ...categories.map(
                          (c) => DropdownMenuItem(value: c.id, child:
Text(c.categoryName)),
```

```
                        ),
                    ],
                    onChanged: (val) => setSheetState(() => tempCategoryId =
val),
                  ),
                  SwitchListTile(
                    contentPadding: EdgeInsets.zero,
                    title: const Text('Featured Only'),
                    value: tempFeatured,
                    onChanged: (val) => setSheetState(() => tempFeatured = val),
                  ),
                  DropdownButtonFormField<String?>(
                    value: tempStock,
                    decoration: const InputDecoration(labelText: 'Stock Status'),
                    items: const [
                      DropdownMenuItem(value: null, child: Text('All')),
                      DropdownMenuItem(value: 'in_stock', child: Text('In Stock
(stock > 0)')),
                      DropdownMenuItem(value: 'low_stock', child: Text('Low Stock
(stock < 5)')),
                      DropdownMenuItem(value: 'out_of_stock', child: Text('Out of
Stock (stock = 0)')),
                    ],
                    onChanged: (val) => setSheetState(() => tempStock = val),
                  ),
                  const SizedBox(height: 8),
                  DropdownButtonFormField<String?>(
                    value: tempRam,
                    decoration: const InputDecoration(labelText: 'RAM'),
                    items: const [
                      DropdownMenuItem(value: null, child: Text('All RAM')),
                      DropdownMenuItem(value: '8', child: Text('8 GB')),
                      DropdownMenuItem(value: '16', child: Text('16 GB')),
                      DropdownMenuItem(value: '32+', child: Text('32 GB or
above')),
                    ],
                    onChanged: (val) => setSheetState(() => tempRam = val),
                  ),
                  const SizedBox(height: 8),
                  DropdownButtonFormField<String?>(
                    value: tempStorage,
                    decoration: const InputDecoration(labelText: 'Storage'),
                    items: const [
                      DropdownMenuItem(value: null, child: Text('All Storage')),
                      DropdownMenuItem(value: '256', child: Text('256 GB')),
                      DropdownMenuItem(value: '512', child: Text('512 GB')),
```

```
                      DropdownMenuItem(value: '1024', child: Text('1 TB (1024
GB)')),
                      DropdownMenuItem(value: '2048+', child: Text('2 TB+ (2048+
GB)')),
                    ],
                    onChanged: (val) => setSheetState(() => tempStorage = val),
                  ),
                  const SizedBox(height: 8),
                  DropdownButtonFormField<String?>(
                    value: tempPrice,
                    decoration: const InputDecoration(labelText: 'Price Range'),
                    items: const [
                      DropdownMenuItem(value: null, child: Text('All Prices')),
                      DropdownMenuItem(value: '<500', child: Text('Less than
500')),
                      DropdownMenuItem(value: '500-1000', child: Text('500 -
1000')),
                      DropdownMenuItem(value: '>1000', child: Text('Greater than
1000')),
                    ],
                    onChanged: (val) => setSheetState(() => tempPrice = val),
                  ),
                  const SizedBox(height: 16),
                  SizedBox(
                    width: double.infinity,
                    child: ElevatedButton(
                      onPressed: () {
                        notifier.applyFilters(
                          brandId: tempBrandId,
                          categoryId: tempCategoryId,
                          featured: tempFfeatured,
                          stock: tempStock,
                          ram: tempRam,
                          storage: tempStorage,
                          price: tempPrice,
                        );
                        Navigator.pop(ctx);
                      },
                      child: const Text('Apply Filters'),
                    ),
                  ),
                ],
              ),
            ),
          );
        },
```

```
      );
    },
  );
}

// Modal chọn Sort (Task 5.4)
Future<void> _openSortDialog() async {
  final notifier = ref.read(computerProvider.notifier);
  final options = <String, String>{
    'price_asc': 'Price: Ascending',
    'price_desc': 'Price: Descending',
    'sold_asc': 'Sold Quantity: Ascending',
    'sold_desc': 'Sold Quantity: Descending',
    'name_asc': 'Product Name: A → Z',
    'name_desc': 'Product Name: Z → A',
    'brand_asc': 'Brand Name: A → Z',
    'brand_desc': 'Brand Name: Z → A',
    'stock_asc': 'Stock: Low → High',
    'stock_desc': 'Stock: High → Low',
  };

  await showModalBottomSheet(
    context: context,
    builder: (ctx) {
      return ListView(
        shrinkWrap: true,
        children: [
          const Padding(
            padding: EdgeInsets.all(16.0),
            child: Text(
              'Sort Computers',
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ),
          ),
          ...options.entries.map(
            (entry) => RadioListTile<String>(
              title: Text(entry.value),
              value: entry.key,
              groupValue: notifier.sortBy,
              onChanged: (val) {
                notifier.setSortOption(val);
                Navigator.pop(ctx);
              },
            ),
          ),
        ],
```

```
        );
      },
    );
  }

  @override
  Widget build(BuildContext context) {
    final computersAsync = ref.watch(computerProvider);

    return Scaffold(
      // 1. AppBar
      appBar: AppBar(title: const Text('Computer Store System')),
      body: Column(
        children: [
          // 2. Promotional Banner (Task 2.6)
          Padding(
            padding: const EdgeInsets.fromLTRB(12, 8, 12, 4),
            child: ClipRRect(
              borderRadius: BorderRadius.circular(12),
              child: AspectRatio(
                aspectRatio: 16 / 5,
                child: Image.asset(
                  'assets/images/promo_banner.png',
                  width: double.infinity,
                  fit: BoxFit.cover,
                  errorBuilder: (context, error, stackTrace) {
                    // Fallback hiển thị banner đẹp nếu chưa copy file ảnh vào
assets/images/
                    return Container(
                      width: double.infinity,
                      decoration: const BoxDecoration(
                        gradient: LinearGradient(
                          colors: [Colors.indigo, Colors.blueAccent],
                        ),
                      ),
                      padding: const EdgeInsets.all(16),
                      child: const Row(
                        children: [
                          Icon(Icons.laptop_mac, color: Colors.white, size: 42),
                          SizedBox(width: 12),
                          Expanded(
                            child: Column(
                              mainAxisAlignment: MainAxisAlignment.center,
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
                                Text(
```

```
                                  'BACK TO SCHOOL SALE UP TO 30% OFF',
                                  style: TextStyle(
                                    color: Colors.white,
                                    fontWeight: FontWeight.bold,
                                    fontSize: 15,
                                  ),
                                ),
                                Text(
                                  'High-Performance Laptops, Desktops &
Components',
                                  style: TextStyle(color: Colors.white70, fontSize:
12),
                                ),
                              ],
                            ),
                          ),
                        ),
                      ],
                    ),
                  );
                },
              ),
            ),
          ),
        ),

        // 3. Search Bar (Task 5.1)
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
          child: TextField(
            controller: _searchCtrl,
            decoration: const InputDecoration(
              hintText: 'Search by Product Name, Brand, or CPU...',
              prefixIcon: Icon(Icons.search),
              isDense: true,
              border: OutlineInputBorder(),
            ),
            onChanged: (val) {
              ref.read(computerProvider.notifier).setSearchKeyword(val);
            },
          ),
        ),

        // 4. Filter Button & 5. Sort Button + Reset Filters (Task 5.2 - 5.5)
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
          child: Row(
```

```
              children: [
                Expanded(
                  child: OutlinedButton.icon(
                    onPressed: _openFilterSheet,
                    icon: const Icon(Icons.filter_list),
                    label: const Text('Filter'),
                  ),
                ),
                const SizedBox(width: 8),
                Expanded(
                  child: OutlinedButton.icon(
                    onPressed: _openSortDialog,
                    icon: const Icon(Icons.sort),
                    label: const Text('Sort'),
                  ),
                ),
                const SizedBox(width: 8),
                TextButton.icon(
                  onPressed: () {
                    _searchCtrl.clear();
                    ref.read(computerProvider.notifier).resetAllFilters();
                  },
                  icon: const Icon(Icons.refresh),
                  label: const Text('Reset Filters'),
                ),
              ],
            ),
          ),

          // 6. Computer Product List (Cuộn độc lập + Pull-to-Refresh)
          Expanded(
            child: computersAsync.when(
              data: (computers) => RefreshIndicator(
                onRefresh: () async {
                  ref.invalidate(computerProvider);
                  await ref.read(computerProvider.future);
                },
                child: computers.isEmpty
                    ? ListView(
                        children: const [
                          SizedBox(height: 100),
                          Center(child: Text('No matching computer products
found.')),
                        ],
                      )
                    : ListView.builder(
```

```
                        itemCount: computers.length,
                        itemBuilder: (context, index) {
                          final comp = computers[index];
                          return ComputerCardItem(
                            computer: comp,
                            onTap: () {
                              Navigator.push(
                                context,
                                MaterialPageRoute(
                                  builder: (_) => ComputerDetailScreen(computerId:
comp.id!),
                                ),
                              );
                            },
                          );
                        },
                      ),
                ),
                error: (err, stack) => Center(child: Text('Error loading products:
$err')),
                loading: () => const Center(child: CircularProgressIndicator()),
              ),
            ),
          ],
        ),
      );
  }
}

class ComputerCardItem extends StatelessWidget {
  final Computer computer;
  final VoidCallback onTap;

  const ComputerCardItem({
    super.key,
    required this.computer,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(12),
```

```
        child: Padding(
          padding: const EdgeInsets.all(12.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  Expanded(
                    child: Text(
                      computer.productName,
                      style: const TextStyle(fontSize: 17, fontWeight:
FontWeight.bold),
                    ),
                  ),
                  if (computer.isFeatured)
                    const Row(
                      children: [
                        Icon(Icons.star, color: Colors.amber, size: 20),
                        SizedBox(width: 4),
                        Text(
                          'Featured',
                          style: TextStyle(
                            color: Colors.amber,
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                      ],
                    ),
                ],
              ),
              const Divider(),
              Text('Brand       : ${computer.brandName ?? ''}'),
              Text('Category    : ${computer.categoryName ?? ''}'),
              Text('CPU         : ${computer.cpu}'),
              Text('RAM         : ${computer.ram} GB'),
              Text('Storage     : ${computer.storage} GB'),
              Text(
                'Price       : \$${computer.price.toStringAsFixed(2)}',
                style: const TextStyle(fontWeight: FontWeight.w600, color:
Colors.green),
              ),
              Text('Stock       : ${computer.stock}'),
              Text('Sold        : ${computer.soldQuantity}'),
            ],
          ),
```

```
        ),
      ),
    );
  }
}
```

5.5. Tab 2 & Màn hình Edit — Thêm / Sửa Computer `lib/ui/computer_form_screen.dart` (Task 4.2, 4.3, 4.6)

Đáp ứng toàn bộ yêu cầu về Widget & Validation của Task 4.2, 4.3, 4.6:

- Load `DropdownButtonFormField` cho `Brand` từ bảng `brands` và `Category` từ bảng `categories`.
- `Switch` cho `Featured`.
- Validation:
  - Tất cả các trường không được để trống.
  - `Product Name` phải là duy nhất (`isProductNameDuplicate`).
  - `Price > 0`.
  - `RAM` từ `4` đến `128` GB.
  - `Storage` từ `128` đến `4096` GB.
  - `Stock >= 0`.
  - `Sold Quantity >= 0`.
- Sau khi thêm mới: hiện Snackbar `"Computer added successfully."` và tự động quay về Store Screen.

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../db/database_helper.dart';
import '../model/computer.dart';
import '../providers/brand_notifier.dart';
import '../providers/category_notifier.dart';
import '../providers/computer_notifier.dart';

class ComputerFormScreen extends ConsumerStatefulWidget {
  final Computer? existingComputer;
  final VoidCallback? onSavedSuccess;

  const ComputerFormScreen({
    super.key,
    this.existingComputer,
    this.onSavedSuccess,
  });

  @override
  ConsumerState<ComputerFormScreen> createState() => _ComputerFormScreenState();
}

class _ComputerFormScreenState extends ConsumerState<ComputerFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _nameCtrl;
  late final TextEditingController _cpuCtrl;
  late final TextEditingController _ramCtrl;
  late final TextEditingController _storageCtrl;
  late final TextEditingController _priceCtrl;
  late final TextEditingController _stockCtrl;
  late final TextEditingController _soldCtrl;

  int? _selectedBrandId;
  int? _selectedCategoryId;
  bool _isFeatured = false;
  String? _duplicateNameError;

  @override
  void initState() {
    super.initState();
    final comp = widget.existingComputer;
    _nameCtrl = TextEditingController(text: comp?.productName ?? '');
    _cpuCtrl = TextEditingController(text: comp?.cpu ?? '');
    _ramCtrl = TextEditingController(text: comp != null ? comp.ram.toString() :
'');
```

```
    _storageCtrl = TextEditingController(text: comp != null ?
comp.storage.toString() : '');
    _priceCtrl = TextEditingController(text: comp != null ? comp.price.toString() :
'');
    _stockCtrl = TextEditingController(text: comp != null ? comp.stock.toString() :
'');
    _soldCtrl = TextEditingController(text: comp != null ?
comp.soldQuantity.toString() : '');
    _selectedBrandId = comp?.brandId;
    _selectedCategoryId = comp?.categoryId;
    _isFeatured = comp?.isFeatured ?? false;
  }

  @override
  void dispose() {
    _nameCtrl.dispose();
    _cpuCtrl.dispose();
    _ramCtrl.dispose();
    _storageCtrl.dispose();
    _priceCtrl.dispose();
    _stockCtrl.dispose();
    _soldCtrl.dispose();
    super.dispose();
  }

  Future<void> _submitForm() async {
    setState(() => _duplicateNameError = null);
    if (!_formKey.currentState!.validate()) return;

    final isDup = await DatabaseHelper.instance.isProductNameDuplicate(
      _nameCtrl.text.trim(),
      excludeId: widget.existingComputer?.id,
    );
    if (isDup) {
      setState(() {
        _duplicateNameError = 'Product Name must be unique.';
      });
      return;
    }

    final computer = Computer(
      id: widget.existingComputer?.id,
      productName: _nameCtrl.text.trim(),
      brandId: _selectedBrandId!,
      categoryId: _selectedCategoryId!,
      cpu: _cpuCtrl.text.trim(),
      ram: int.parse(_ramCtrl.text.trim()),
```

```
      storage: int.parse(_storageCtrl.text.trim()),
      price: double.parse(_priceCtrl.text.trim()),
      stock: int.parse(_stockCtrl.text.trim()),
      soldQuantity: int.parse(_soldCtrl.text.trim()),
      isFeatured: _isFeatured,
    );

    if (widget.existingComputer == null) {
      await ref.read(computerProvider.notifier).addComputer(computer);
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Computer added successfully.')),
      );
      if (widget.onSavedSuccess != null) {
        widget.onSavedSuccess!();
      } else {
        Navigator.pop(context, true);
      }
    } else {
      await ref.read(computerProvider.notifier).updateComputer(computer);
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Computer updated successfully.')),
      );
      Navigator.pop(context, true);
    }
  }

  @override
  Widget build(BuildContext context) {
    final brandsAsync = ref.watch(brandProvider);
    final categoriesAsync = ref.watch(categoryProvider);
    final isEditing = widget.existingComputer != null;

    return Scaffold(
      appBar: AppBar(
        title: Text(isEditing ? 'Edit Computer' : 'Add Computer'),
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16.0),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _nameCtrl,
```

```
                decoration: InputDecoration(
                  labelText: 'Product Name',
                  border: const OutlineInputBorder(),
                  errorText: _duplicateNameError,
                ),
                validator: (val) =>
                    val == null || val.trim().isEmpty ? 'Product Name is required'
: null,
              ),
              const SizedBox(height: 12),

              // Brand Dropdown
              brandsAsync.when(
                data: (brands) => DropdownButtonFormField<int>(
                  value: brands.any((b) => b.id == _selectedBrandId) ?
_selectedBrandId : null,
                  decoration: const InputDecoration(
                    labelText: 'Brand',
                    border: OutlineInputBorder(),
                  ),
                  items: brands
                      .map((b) => DropdownMenuItem<int>(
                            value: b.id,
                            child: Text(b.brandName),
                          ))
                      .toList(),
                  onChanged: (val) => setState(() => _selectedBrandId = val),
                  validator: (val) => val == null ? 'Brand is required' : null,
                ),
                error: (e, _) => Text('Error loading brands: $e'),
                loading: () => const LinearProgressIndicator(),
              ),
              const SizedBox(height: 12),

              // Category Dropdown
              categoriesAsync.when(
                data: (categories) => DropdownButtonFormField<int>(
                  value: categories.any((c) => c.id == _selectedCategoryId)
                      ? _selectedCategoryId
                      : null,
                  decoration: const InputDecoration(
                    labelText: 'Category',
                    border: OutlineInputBorder(),
                  ),
                  items: categories
                      .map((c) => DropdownMenuItem<int>(
```

```
                value: c.id,
                child: Text(c.categoryName),
              ))
          .toList(),
      onChanged: (val) => setState(() => _selectedCategoryId = val),
      validator: (val) => val == null ? 'Category is required' : null,
    ),
    error: (e, _) => Text('Error loading categories: $e'),
    loading: () => const LinearProgressIndicator(),
  ),
  const SizedBox(height: 12),

  TextFormField(
    controller: _cpuCtrl,
    decoration: const InputDecoration(
      labelText: 'CPU',
      border: OutlineInputBorder(),
    ),
    validator: (val) =>
        val == null || val.trim().isEmpty ? 'CPU is required' : null,
  ),
  const SizedBox(height: 12),

  TextFormField(
    controller: _ramCtrl,
    keyboardType: TextInputType.number,
    decoration: const InputDecoration(
      labelText: 'RAM (4 – 128 GB)',
      border: OutlineInputBorder(),
    ),
    validator: (val) {
      if (val == null || val.trim().isEmpty) return 'RAM is required';
      final ram = int.tryParse(val.trim());
      if (ram == null || ram < 4 || ram > 128) {
        return 'RAM must be between 4 GB and 128 GB';
      }
      return null;
    },
  ),
  const SizedBox(height: 12),

  TextFormField(
    controller: _storageCtrl,
    keyboardType: TextInputType.number,
    decoration: const InputDecoration(
      labelText: 'Storage (128 – 4096 GB)',
```

```
                  border: OutlineInputBorder(),
                ),
                validator: (val) {
                  if (val == null || val.trim().isEmpty) return 'Storage is
required';
                  final storage = int.tryParse(val.trim());
                  if (storage == null || storage < 128 || storage > 4096) {
                    return 'Storage must be between 128 GB and 4096 GB';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 12),

              TextFormField(
                controller: _priceCtrl,
                keyboardType: const TextInputType.numberWithOptions(decimal: true),
                decoration: const InputDecoration(
                  labelText: 'Price (\$)',
                  border: OutlineInputBorder(),
                ),
                validator: (val) {
                  if (val == null || val.trim().isEmpty) return 'Price is
required';
                  final price = double.tryParse(val.trim());
                  if (price == null || price <= 0) {
                    return 'Price must be > 0';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 12),

              TextFormField(
                controller: _stockCtrl,
                keyboardType: TextInputType.number,
                decoration: const InputDecoration(
                  labelText: 'Stock',
                  border: OutlineInputBorder(),
                ),
                validator: (val) {
                  if (val == null || val.trim().isEmpty) return 'Stock is
required';
                  final stock = int.tryParse(val.trim());
                  if (stock == null || stock < 0) {
                    return 'Stock must be >= 0';
```

```
                }
                return null;
              },
            ),
            const SizedBox(height: 12),

            TextFormField(
              controller: _soldCtrl,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(
                labelText: 'Sold Quantity',
                border: OutlineInputBorder(),
              ),
              validator: (val) {
                if (val == null || val.trim().isEmpty) return 'Sold Quantity is
required';
                final sold = int.tryParse(val.trim());
                if (sold == null || sold < 0) {
                  return 'Sold Quantity must be >= 0';
                }
                return null;
              },
            ),
            const SizedBox(height: 8),

            SwitchListTile(
              title: const Text('Featured Product'),
              value: _isFeatured,
              onChanged: (val) => setState(() => _isFeatured = val),
            ),
            const SizedBox(height: 16),

            SizedBox(
              width: double.infinity,
              height: 48,
              child: ElevatedButton(
                onPressed: _submitForm,
                child: Text(isEditing ? 'Update Computer' : 'Save Computer'),
              ),
            ),
          ],
        ),
      ),
    ),
  );
}
```

```
}
```

5.6. Màn hình Chi Tiết Sản Phẩm `lib/ui/computer_detail_screen.dart` (Task 4.3, 4.4, 4.5)

- Truy vấn thông tin chi tiết bằng `DatabaseHelper.instance.getComputerById(computerId)` (có `INNER JOIN`).
- Hiển thị đầy đủ 10 trường + 3 giá trị tính toán trực tiếp trong Flutter:
  - `Revenue = Price × Sold Quantity`
  - `Inventory Value = Price × Stock`
  - `Total Units = Stock + Sold Quantity`
- Có nút **Edit** (chuyển sang `ComputerFormScreen`) và nút **Delete** (mở `AlertDialog` `"Are you sure you want to delete this product?"` với 2 nút `Cancel` và `Delete`, xóa xong hiện `"Computer deleted successfully."` và `Navigator.pop` về Store Screen).

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../db/database_helper.dart';
import '../model/computer.dart';
import '../providers/computer_notifier.dart';
import 'computer_form_screen.dart';

class ComputerDetailScreen extends ConsumerStatefulWidget {
  final int computerId;

  const ComputerDetailScreen({super.key, required this.computerId});

  @override
  ConsumerState<ComputerDetailScreen> createState() =>
_ComputerDetailScreenState();
}

class _ComputerDetailScreenState extends ConsumerState<ComputerDetailScreen> {
  Future<Computer?>? _computerFuture;

  @override
  void initState() {
    super.initState();
    _loadDetail();
  }

  void _loadDetail() {
    setState(() {
      _computerFuture = DatabaseHelper.instance.getComputerById(widget.computerId);
    });
  }

  Future<void> _confirmDelete(Computer computer) async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Delete Product'),
        content: const Text('Are you sure you want to delete this product?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: const Text('Cancel'),
          ),
          ElevatedButton(
            style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
```

```
            onPressed: () => Navigator.pop(ctx, true),
            child: const Text('Delete', style: TextStyle(color: Colors.white)),
          ),
        ],
      ),
    );

    if (confirmed == true) {
      await ref.read(computerProvider.notifier).deleteComputer(computer.id!);
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Computer deleted successfully.')),
      );
      Navigator.pop(context); // Return to Store Screen
    }
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<Computer?>(
      future: _computerFuture,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Scaffold(body: Center(child: CircularProgressIndicator()));
        }
        final comp = snapshot.data;
        if (comp == null) {
          return Scaffold(
            appBar: AppBar(title: const Text('Computer Detail')),
            body: const Center(child: Text('Product not found.')),
          );
        }

        return Scaffold(
          appBar: AppBar(
            title: Text(comp.productName),
            actions: [
              IconButton(
                icon: const Icon(Icons.edit),
                onPressed: () async {
                  final updated = await Navigator.push<bool>(
                    context,
                    MaterialPageRoute(
                      builder: (_) => ComputerFormScreen(existingComputer: comp),
                    ),
                  );
```

```
                if (updated == true) {
                  _loadDetail();
                }
              },
            ),
            IconButton(
              icon: const Icon(Icons.delete, color: Colors.redAccent),
              onPressed: () => _confirmDelete(comp),
            ),
          ],
        ),
        body: SingleChildScrollView(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Card(
                child: Padding(
                  padding: const EdgeInsets.all(16.0),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      const Text(
                        'Product Specifications',
                        style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold),
                      ),
                      const Divider(),
                      _buildRow('Product Name', comp.productName),
                      _buildRow('Brand', comp.brandName ?? ''),
                      _buildRow('Category', comp.categoryName ?? ''),
                      _buildRow('CPU', comp.cpu),
                      _buildRow('RAM', '${comp.ram} GB'),
                      _buildRow('Storage', '${comp.storage} GB'),
                      _buildRow('Price', '\$${comp.price.toStringAsFixed(2)}'),
                      _buildRow('Stock', '${comp.stock}'),
                      _buildRow('Sold Quantity', '${comp.soldQuantity}'),
                      _buildRow('Featured', comp.isFeatured ? 'Yes (★)' : 'No'),
                    ],
                  ),
                ),
              ),
              const SizedBox(height: 16),
              Card(
                color: Colors.indigo.shade50,
                child: Padding(
```

```
                    padding: const EdgeInsets.all(16.0),
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        const Text(
                          'Business Calculations (Calculated in Flutter)',
                          style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold),
                        ),
                        const Divider(),
                        _buildRow(
                          'Revenue (Price × Sold)',
                          '\$${comp.revenue.toStringAsFixed(2)}',
                        ),
                        _buildRow(
                          'Inventory Value (Price × Stock)',
                          '\$${comp.inventoryValue.toStringAsFixed(2)}',
                        ),
                        _buildRow(
                          'Total Units (Stock + Sold)',
                          '${comp.totalUnits} units',
                        ),
                      ],
                    ),
                  ),
                ),
              ],
            ),
          ),
        );
      },
    );
  }

  Widget _buildRow(String label, String value) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6.0),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Text(label, style: const TextStyle(fontWeight: FontWeight.w500)),
          Text(value, style: const TextStyle(fontWeight: FontWeight.bold)),
        ],
      ),
    );
  }
```

```
}
```

5.7. Tab 5 — Màn hình Thống Kê `lib/ui/statistics_screen.dart` (Task 6.1 → 6.8)

Sử dụng `ref.watch(statisticsProvider)` kết hợp `.when(data: ..., error: ..., loading: ...)` giống hệt để hiển thị toàn bộ 8 nhóm thống kê từ SQLite Aggregate Functions:

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/statistics_notifier.dart';

class StatisticsScreen extends ConsumerWidget {
  const StatisticsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final statsAsync = ref.watch(statisticsProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Business Statistics')),
      body: statsAsync.when(
        data: (stats) => RefreshIndicator(
          onRefresh: () async {
            ref.invalidate(statisticsProvider);
            await ref.read(statisticsProvider.future);
          },
          child: ListView(
            padding: const EdgeInsets.all(12.0),
            children: [
              // 6.1 General Statistics
              _sectionTitle('6.1 General Statistics'),
              Card(
                child: Padding(
                  padding: const EdgeInsets.all(12.0),
                  child: Column(
                    children: [
                      _statRow('Total Products', '${stats.general.totalProducts}'),
                      _statRow('Total Revenue',
'\$${stats.general.totalRevenue.toStringAsFixed(2)}'),
                      _statRow('Total Sold Quantity',
'${stats.general.totalSoldQuantity}'),
                      _statRow('Average Product Price',
'\$${stats.general.averagePrice.toStringAsFixed(2)}'),
                      _statRow('Total Inventory Value',
'\$${stats.general.totalInventoryValue.toStringAsFixed(2)}'),
                    ],
                  ),
                ),
              ),
              const SizedBox(height: 12),

              // 6.4 Best Selling, 6.5 Most Expensive, 6.7 Top Brand
```

```
              _sectionTitle('Highlights (6.4, 6.5, 6.7)'),
              Card(
                child: Padding(
                  padding: const EdgeInsets.all(12.0),
                  child: Column(
                    children: [
                      _statRow(
                        'Best Selling Product',
                        stats.bestSellingProduct != null
                            ? '${stats.bestSellingProduct!['product_name']}
(${stats.bestSellingProduct!['sold_quantity']} sold)'
                            : 'N/A',
                      ),
                      _statRow(
                        'Most Expensive Product',
                        stats.mostExpensiveProduct != null
                            ? '${stats.mostExpensiveProduct!['product_name']}
(\$${stats.mostExpensiveProduct!['price']})'
                            : 'N/A',
                      ),
                      _statRow(
                        'Top Brand (Highest Sold)',
                        stats.topBrand != null
                            ? '${stats.topBrand!['brand_name']} (${stats.topBrand!
['TotalSold']} sold)'
                            : 'N/A',
                      ),
                    ],
                  ),
                ),
              ),
              const SizedBox(height: 12),

              // 6.8 Featured Statistics
              _sectionTitle('6.8 Featured Statistics'),
              Card(
                child: Padding(
                  padding: const EdgeInsets.all(12.0),
                  child: Column(
                    children: [
                      _statRow('Total Featured Products',
'${stats.featuredStats.totalFeaturedProducts}'),
                      _statRow('Total Featured Revenue',
'\$${stats.featuredStats.totalFeaturedRevenue.toStringAsFixed(2)}'),
                      _statRow('Total Featured Sold Quantity',
'${stats.featuredStats.totalFeaturedSoldQuantity}'),
```

```
                    ],
                  ),
                ),
              ),
              const SizedBox(height: 12),

              // 6.2 Statistics by Brand
              _sectionTitle('6.2 Statistics by Brand'),
              ...stats.byBrand.map(
                (b) => Card(
                  child: ListTile(
                    title: Text(b.brandName, style: const TextStyle(fontWeight:
FontWeight.bold)),
                    subtitle: Text(
                      'Products: ${b.totalProducts} | Sold: ${b.totalSold}\n'
                      'Revenue: \$${b.revenue.toStringAsFixed(2)} | Inventory:
\$${b.inventoryValue.toStringAsFixed(2)}',
                    ),
                  ),
                ),
              ),
              const SizedBox(height: 12),

              // 6.3 Statistics by Category
              _sectionTitle('6.3 Statistics by Category'),
              ...stats.byCategory.map(
                (c) => Card(
                  child: ListTile(
                    title: Text(c.categoryName, style: const TextStyle(fontWeight:
FontWeight.bold)),
                    subtitle: Text(
                      'Total Products: ${c.totalProducts} | Total Sold:
${c.totalSold}\n'
                      'Total Revenue: \$${c.revenue.toStringAsFixed(2)}',
                    ),
                  ),
                ),
              ),
              const SizedBox(height: 12),

              // 6.6 Low Stock Products (stock < 5)
              _sectionTitle('6.6 Low Stock Products (Stock < 5)'),
              if (stats.lowStockProducts.isEmpty)
                const Card(
                  child: Padding(
                    padding: EdgeInsets.all(12.0),
```

```
                    child: Text('No low stock products.'),
                  ),
                )
              else
                ...stats.lowStockProducts.map(
                  (item) => Card(
                    child: ListTile(
                      leading: const Icon(Icons.warning_amber_rounded, color:
Colors.orange),
                      title: Text('${item['product_name']}'),
                      subtitle: Text('Brand: ${item['brand_name']}'),
                      trailing: Text(
                        'Stock: ${item['stock']}',
                        style: const TextStyle(color: Colors.red, fontWeight:
FontWeight.bold),
                      ),
                    ),
                  ),
                ),
            ],
          ),
        ),
        error: (err, stack) => Center(child: Text('Error loading statistics:
$err')),
        loading: () => const Center(child: CircularProgressIndicator()),
      ),
    );
  }

  Widget _sectionTitle(String text) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6.0, horizontal: 4.0),
      child: Text(
        text,
        style: const TextStyle(fontSize: 17, fontWeight: FontWeight.bold, color:
Colors.indigo),
      ),
    );
  }

  Widget _statRow(String label, String value) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 4.0),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
```

```
          Text(label),
          Text(value, style: const TextStyle(fontWeight: FontWeight.bold)),
        ],
      ),
    );
  }
}
```

5.8. Tab 6 — Màn hình Sản Phẩm Nổi Bật `lib/ui/featured_screen.dart`

Hiển thị danh sách các sản phẩm có `is_featured = 1` và cho phép bấm vào xem `ComputerDetailScreen`:

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../db/database_helper.dart';
import '../model/computer.dart';
import '../providers/computer_notifier.dart';
import 'computer_detail_screen.dart';
import 'store_screen.dart';

class FeaturedScreen extends ConsumerWidget {
  const FeaturedScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Lắng nghe computerProvider để tự động rebuild khi sản phẩm thay đổi
    ref.watch(computerProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Featured Products')),
      body: FutureBuilder<List<Computer>>(
        future: DatabaseHelper.instance.filterComputers(featuredOnly: true),
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          }
          final featuredList = snapshot.data ?? [];
          if (featuredList.isEmpty) {
            return const Center(child: Text('No featured products available.'));
          }
          return ListView.builder(
            itemCount: featuredList.length,
            itemBuilder: (context, index) {
              final comp = featuredList[index];
              return ComputerCardItem(
                computer: comp,
                onTap: () {
                  Navigator.push(
                    context,
                    MaterialPageRoute(
                      builder: (_) => ComputerDetailScreen(computerId: comp.id!),
                    ),
                  );
                },
```

```
              );
            },
          );
        },
      ),
    );
  }
}
```

Bước 6: Cập nhật `lib/main.dart`

Giải thích

Giữ nguyên cách khởi tạo `WidgetsFlutterBinding.ensureInitialized()` và `ProviderScope(child: MyApp())` của , trỏ `home:` sang `MainNavigationScreen()`.

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'ui/main_navigation_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: MyApp()));
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'ComputerStoreSys',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
        useMaterial3: true,
      ),
      home: const MainNavigationScreen(),
    );
  }
}
```

Bước 7: Trình tự thực thi & Kiểm thử (Verification Checklist)

1. **Tạo và cập nhật các file** theo đúng thứ tự Bước 1 → Bước 6.
2. **Chạy sinh code Riverpod**:

   ```
   bash
   dart run build_runner build --delete-conflicting-outputs
   ```

3. **Kiểm tra phân tích tĩnh**:

```
bash
flutter analyze
```

4. **Chạy ứng dụng** trên Android Emulator hoặc Chrome:

```
bash
flutter run
```

5. **Danh sách kiểm thử đầy đủ theo đề bài**:
    - **Tab Store**: Hiện Banner ở trên cùng -> Search Bar -> Filter/Sort/Reset -> Danh sách 15 sản phẩm mẫu. Thử gõ `"ASUS"` vào ô Search, thử lọc `RAM = 16 GB` + `Price = 500–1000`, thử bấm `Reset Filters`.
    - **Tab Add Computer**: Thử để trống, nhập RAM < 4, Price <= 0 hoặc trùng tên sản phẩm để kiểm tra thông báo lỗi; thêm sản phẩm hợp lệ và kiểm tra Snackbar `"Computer added successfully."` cùng việc tự động chuyển về Tab Store.
    - **Computer Detail**: Bấm vào 1 sản phẩm trên Store, kiểm tra 3 chỉ số `Revenue`, `Inventory Value`, `Total Units`, thử sửa và xóa sản phẩm.
    - **Tab Brands & Categories**: Kiểm tra số lượng sản phẩm đi kèm mỗi Brand/Category (`LEFT JOIN`), thử xóa Brand/Category đang có sản phẩm (phải chặn và hiện đúng thông báo lỗi), thử tạo mới Brand/Category rồi xóa.
    - **Tab Statistics & Tab Featured**: Kiểm tra số liệu tổng hợp tự động cập nhật ngay sau khi thêm/sửa/xóa máy tính.